

FuMi: A Runtime Fuzz-based Machine Learning Precision Measurement and Testing Framework

PENG ZHANG and MIKE PAPADAKIS, Interdisciplinary Centre for Security, Reliability and Trust, University of Luxembourg, Esch-sur-Alzette, Luxembourg
YUMING ZHOU, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

The rapid evolution of machine learning model training has outpaced the development of corresponding measurement and testing tools, leading to two significant challenges. Firstly, developers of deep learning frameworks struggle to identify operators that fail to meet precision criteria, as these issues may only manifest in a few data points. Secondly, model trainers lack effective methods to estimate precision loss caused by operators during training. To address these issues, we introduce a Pythonic framework inspired by common network layer definitions in deep learning. Our framework includes two new layers: the Fuzz Layer and the Check Layer, designed to aid in measurement and testing. The Fuzz Layer introduces minor perturbations to tensor inputs for any deterministic layer under testing (LUT). The Check Layer then measures precision by analyzing the differences before and after the perturbation. This approach estimates a lower bound of the maximal relative error and alerts developers or trainers of potential bugs if the difference exceeds a pre-defined tolerance. Additionally, Check Layers can be used independently to conduct precision tests for specific layers, ensuring the precision of operators during runtime. Despite the additional memory and time requirements, this runtime testing ensures proper training of the original model. We demonstrate the utility of our framework, FuMi, through two experiments. First, we tested 21 torch operators across nine popular machine learning models using PyTorch for various tasks, finding that the conv2d and linear operators often fail to meet precision requirements. Second, to showcase the generalizability of our framework, we tested the ATTENTION operator. By comparing different implementations of state-of-the-art ATTENTION operators, we found that the maximum relative error of the ATTENTION operator is not less than 1%, which is 13 times larger than that measured by Predoo (a unit test tool). This framework provides a robust solution for identifying precision issues in deep learning models, ensuring reliable and accurate model training.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**;

Additional Key Words and Phrases: Deep learning, fuzzing testing, software measurement

The work in this article is supported partially by the National Natural Science Foundation of China (62172205).

Authors' Contact Information: Peng Zhang (corresponding author), Interdisciplinary Centre for Security, Reliability and Trust, University of Luxembourg, Esch-sur-Alzette, Luxembourg; e-mail: peng.zhang@uni.lu; Mike Papadakis, Interdisciplinary Centre for Security, Reliability and Trust, University of Luxembourg, Esch-sur-Alzette, Luxembourg; e-mail: michail.papadakis@uni.lu; Yuming Zhou, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zhoyuming@nju.edu.cn.



This work is licensed under Creative Commons Attribution International 4.0.

© 2026 Copyright held by the owner/author(s).

ACM 1557-7392/2026/2-ART66

<https://doi.org/10.1145/3734866>

ACM Reference format:

Peng Zhang, Mike Papadakis, and Yuming Zhou. 2026. FuMi: A Runtime Fuzz-based Machine Learning Precision Measurement and Testing Framework. *ACM Trans. Softw. Eng. Methodol.* 35, 3, Article 66 (February 2026), 24 pages.
<https://doi.org/10.1145/3734866>

1 Introduction

With the rapid development of **deep learning (DL)**, the field of AI software engineering is encountering significant challenges. Measuring and testing AI products to manage their quality is becoming an increasingly critical and pressing issue. In this article, we will primarily focus on computational precision. Specifically, we will explore the question: How can we measure the difference between the computed values of a deterministic operator and its theoretical values?

By operator, we refer to functions used in scientific computation, such as `torch.add`. These operators take a floating-point tensor as input and produce a floating-point tensor as output, without involving any randomness.

The error of an operator is influenced by various factors, including:

- (1) *Impact of Computing Chips*: With the rapid advancement of DL, more manufacturers are introducing heterogeneous computing chips [4, 16, 20] to support various DL frameworks (e.g., PyTorch [23], TensorFlow [2], and Keras [3]). These hardware platforms can introduce complexity into the software stack, leading to discrepancies in computational results.
- (2) *Impact of Data Types*: The error between the computed result and the scientific result can vary with different input data types. For example, an issue [5] reported in the famous LLaMA code repository highlighted that using `bf16` instead of `float32` for a certain layer resulted in insufficient computational precision.
- (3) *Impact of Different Algorithms*: Algorithmic differences, such as variations in CUDA libraries [22] or ATTENTION operator implementations [6, 7], often result in computational discrepancies despite mathematical equivalence.

From the issue mentioned in the LLaMA repository, we can see the typical timeline for addressing precision issues: First, developers notice anomalies in the model training process (e.g., lack of convergence or slow convergence); then, using their experience, they identify the problematic operator layer by layer; finally, they find a solution. Therefore, the main challenges lie in locating and fixing operator precision issues. Next, let us explore how we can measure the precision of an operator using existing approaches.

To assess the computational precision of operators, unit testing serves as a fundamental approach. Our examination of PyTorch's open source test cases reveals that most operators only include a single test case for floating-point precision validation, which is insufficient for comprehensive input space coverage. To address this limitation, researchers have developed several enhanced testing methodologies.

First, comparative approaches use CPU results as ground truth to evaluate GPU computation errors. Alternative methods like Predoo [32] employ search algorithms to identify inputs that maximize computational errors. More recently, automated testing frameworks including FreeFuzz [29], VFuzz [30], and DeepREL [8] have emerged, which generate test inputs through either model data analysis or differential testing mechanisms.

However, there are two main drawbacks for these unit testing approaches:

- Manual Definition of Constraints: Constraints between inputs need to be manually defined. For instance, in the ATTENTION operator, inputs Q, K, and V must have the same shape (see Section 2 for more information on the ATTENTION operator).
- Difference between Constructed and Real Inputs: Bugs often occur in edge cases, which may be related to GPU architecture specifics (e.g., whether matrix dimensions are multiples of 8 can trigger different code branches). PyTorch users may not be fully aware of these nuances, and their unit tests may not cover all branches. Thus, even with unit testing, the precision of the operator might not be accurately reflected. Essentially, this is due to the difference between the distribution of constructed inputs and real inputs.

Another approach is runtime data monitoring, saving data for subsequent error analysis. The official CANN documentation [17] provides a dump tool to record the operator's computation results at runtime. Later, we can analyze the relative error by comparing the results from the NPU and GPU. This method also has two main drawbacks:

- Heavy Cost of Data Dump: As described in the official documentation, the files generated during dumping often exceed tens of gigabytes.
- Heavy Reliance on Golden Data: To calculate relative error, we must set up the same model on the GPU or CPU to get golden data. This process is not just a simple code migration. For example, the `flash_attention` operator only runs on GPUs, and its official repository does not provide implementations for other hardware. Thus, while theoretically feasible, this method is practically challenging and costly.

At this point, readers may also realize the difficulty of this problem. This highlights that current software engineering measurement and testing for AI software lag far behind the rapid development of AI itself. To address these challenges, this article introduces a lightweight, golden-data-independent testing framework FuMi based on fuzzing. For operators under test, FuMi can estimate a lower bound of maximal relative error. By adhering to a precision criterion, our tool identifies inputs that do not meet the specified precision during training. This approach involves defining Fuzz Layers and corresponding Check Layers, which perform minor perturbations on tensor inputs for any **layer under testing (LUT)**. As the data exits the LUT post-perturbation, the Check Layer scrutinizes the error before and after the perturbation, estimating the relative error and promptly notifying the developer/trainer of any identified issues.

We now provide a simple example as an introduction to how the FuMi mechanism works. Suppose the reader has a calculator at hand, and in our calculation of " $a + b$ " we get "1." After this, we calculate " $b + a$ " and get "2." Then even if we do not know what the golden data of " $a + b$ " is, we can be sure that at least one of the results differs from the golden data by more than or equal to 0.5. Further, we can reason that the relative error is not less than $1/3$. We will elaborate on the specific details of FuMi in Section 3.

In summary, this work makes the following contributions:

- Fuzzing Testing Framework for Machine Learning: We introduce FuMi which stands for *Fuzzing Testing Framework for Machine Learning* models aimed at detecting layers/operators that fail to meet precision criteria. This approach not only provides a tool for DL framework developers to test operators but also offers a means for users to calculate the precision of operators in the model.
- Error Measurement System without Golden Data: We have developed an error analysis system without relying on golden data. Our analysis focuses on relative errors between the results of

operator calculations and scientific calculations (i.e., ground truth). This approach allows us to mitigate false positives. In the “Improve Precision” mode, we can guarantee the absence of false positives.

- Model Testing and Findings: We have tested 21 different torch operators across nine popular machine learning models using PyTorch for various tasks. We found that the `conv2d` and `linear` operators often fail to meet the given precision requirements. Then, to showcase the generalizability of our framework, we tested the `ATTENTION` operator. By comparing different implementations of state-of-the-art `ATTENTION` operators, we found that the maximum relative error of the `ATTENTION` operator is not less than 1%. Compared with Predoo, FuMi can measure (explore) the error that is 13 times larger.
- Code and Data Availability: Our code and data are publicly available [1].

These contributions collectively advance the understanding and applicability of fuzzing techniques in the domain of DL testing.

The rest of this article is organized as follows. Section 2 introduces a motivating example. Section 3 presents the framework FuMi we proposed. Section 4 describes the settings of experiments. Section 5 reports the experimental results. Section 6 discusses a case study, analyzes the false negatives, and shows the ease of FuMi. Section 7 introduces related work. Section 8 discusses potential threats. Section 9 shows implications for the researchers and practitioners. Section 10 provides a summary of the entire article.

2 Motivating Example

In this section, we present a motivating case study based on our research into the `ATTENTION` operator in the LLaMA model. Readers will gain insight into the necessity of measuring the precision of certain operators within a model, the limitations of existing techniques, and the advantages of our proposed framework, FuMi.

2.1 Necessity of Precision Measurement

First, let us address the need for precision measurement. During model training, high-precision data calculations are crucial. Insufficient precision in an operator can cause the entire training process to fail to converge. An example from the LLaMA code annotations illustrates this issue [5]: The `LLaMARotaryEmbedding` layer, when using `bf16` precision, experiences precision loss, which slows down the model’s convergence. Only by using `float32` precision can we ensure adequate precision for this layer’s outputs. Therefore, to maintain the model’s accuracy and efficiency, we need a methodology and specific technical implementation to support AI software engineering capabilities.

2.2 Limitations of Existing Techniques

Next, let us examine the existing techniques for measuring an operator’s computational precision before FuMi. Existing techniques can be used in two different stages.

2.2.1 Unit Testing. The first stage is unit testing. However, this stage is limited because the input space for machine learning operators is typically high-dimensional and nearly impossible to exhaustively test. For example, in matrix multiplication, the shape of the first matrix can vary widely, and tests are often conducted only on specific shapes. Consequently, the inputs during actual operator execution can differ significantly from those in unit tests. In summary, unit testing is challenging to automate and, even when automated, only provides a biased sample of the operator’s input space, leading to substantial differences from real inputs.

2.2.2 Runtime Testing. In the second stage, we consider methods to capture the real inputs and outputs during runtime and measure their precision. For instance, Huawei Ascend's CANN (similar to CUDA) documentation provides a “dump” solution to address this [17]. This technique collects the input and output data of the object on the NPU being dumped for later comparison with GPU/CPU results. However, this approach has significant drawbacks, especially with large training datasets, as each iteration can generate tens of gigabytes of data, making it impractical for real-world use. On the other hand, it is highly dependent on the ground truth (e.g., CPU's results) to compute the precision of the dumped results.

2.3 Case Study: ATTENTION Operator in Transformers

With the background above, we examine a more specific case: the ATTENTION operator in transformers. The ATTENTION operator, a cornerstone of modern transformer architectures, computes the relationships between elements in a sequence to capture long-range dependencies effectively. This is achieved using three key components: Query (Q), Key (K), and Value (V). These components facilitate the computation of attention scores, which determine the contribution of each element in the sequence to the output. Originally introduced in the seminal work on the Transformer architecture by Vaswani et al. [27], the attention mechanism has since become a fundamental building block for state-of-the-art models across various domains, such as language modeling and **computer vision (CV)**.

Researchers have proposed various optimized implementations, such as flash-attention v1 and v2, claiming significant speed improvements over older algorithms. However, the precision differences between algorithms receive much less attention. Although the new algorithms do not involve any approximations and theoretically should yield identical attention results, numerical computations on current hardware do not always maintain mathematical equivalence. For instance, the commutative property of addition ($a + b + c = a + c + b$) may not hold. Therefore, we focus on this case on how to evaluate the relative error of the ATTENTION operators. *Note that we set “dropout = 0” to ensure that ATTENTION generates deterministic results. For the entire article, all the studied operators are deterministic.*

We then compare Predoo algorithm (a unit testing approach for AI operators involves floating-point computations [32]) and FuMi in the context of this operator. Since the ATTENTION operator is not available on the CPU, dumping tools are unusable. To use Predoo tool, we need to manually set up the features of inputs. Predoo algorithm is essentially a methodology that searches for inputs with the largest relative error via a genetic algorithm. However, when implemented for a specific operator, we have to manually determine features such as the shape of the input. For example, for matrix multiplication, $X \times Y$: the second dimension of X must match the first dimension of Y . This information needs to be determined manually by the developer. In the ATTENTION case, Q, K, and V need to have the same shape. After that, when using Predoo algorithm for unit testing, the maximum relative error found was 0.08%, given the shapes of Q, K, and V matrices ($\text{shape} = (1, 3, 10, 10)$).

2.4 Usage of FuMi on ATTENTION Operator

For FuMi, it is much simpler to use than Predoo algorithm or dump methods. Predoo algorithm requires writing specific test scripts for each operator, while FuMi only needs two additional lines of code, even for custom operators.

For example, consider two implementations of ATTENTION:

```
def attn1(): ...
def attn2(): ...
```

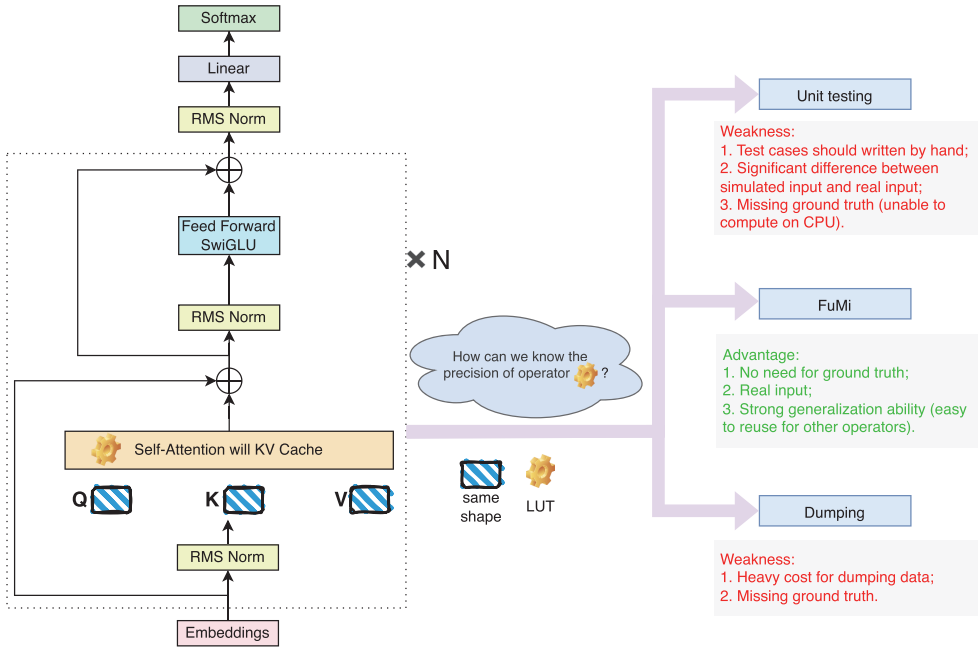


Fig. 1. Comparison of approaches for testing “ATTENTION” precision in LLaMA.

To test with FuMi, simply add:

```
import decorate_function
@decorate_function("attention," attn2)
def attn1(): ...
```

Then run the train script as before, we can get the precision of ATTENTION: We found that the lower bound of the maximum relative error is greater than 1%. This indicates that FuMi can capture significantly higher precision errors with real inputs compared to unit test simulations, providing a more accurate measure of operator precision.

Figure 1 shows this case with the comparison between different approaches. This case study demonstrates that FuMi is both convenient and effective. In the next section, we will delve into the specific approach behind FuMi.

3 Approach

The three subsections within this section elaborate on the Check Layer, Fuzz Layer, and additional options. Through these subsections, we demonstrate that this approach not only possesses the flexibility to be generalized and applied to any operator/layer but also offers the capability to uniquely test specific properties for individual operators. This inherent flexibility empowers designers and users of DL frameworks to extend the approach according to their specific requirements.

At a high level, our test logic follows the principle: “If the input stays the same, then the output should stay the same mathematically.” Figure 2 illustrates the main flow of implementing fuzz testing for the LUT. The yellow segment of the figure depicts the original model for image classification. We strategically insert the Fuzz Layer before and the Check Layer after the LUT, denoted by the purple squares on the left and right, respectively.

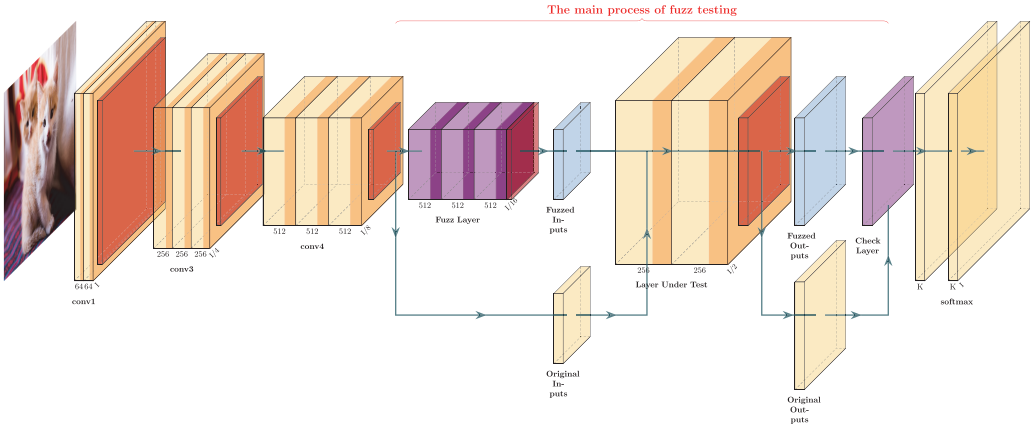


Fig. 2. A DL model with Fuzz Layer and Check Layer inserted.

The Fuzz Layer initiates by perturbing the inputs to generate highly similar inputs for the LUT, represented by the blue squares on the left. Subsequently, both the fuzzed inputs and the original inputs are fed into the LUT, resulting in fuzzed outputs and original outputs, represented by the blue square on the right. Finally, the Check Layer meticulously compares and checks the two outputs. Importantly, the Check Layer returns only the original outputs to the subsequent layer, ensuring that the training of the original model remains unchanged throughout this process. Given the straightforward logic of the Check Layer, we will introduce it first.

3.1 Error Measurement in Check Layer

Based on two outputs, the Check Layer is used to measure the computational precision of the LUT and verify if it meets a given standard of precision. The fundamental principle of our testing approach rests on the concept that “if the input remains the same, the output should remain the same mathematically.” For example, if we use two different algorithms to compute the ATTENTION value under the same group of inputs, then mathematically this result should be equal. Then, we delve into this idea further through the following refinements.

Firstly, we assume that the function the LUT aims to compute mathematically is $f(x)$, and the function the LUT actually computes is $g(x)$. We define the maximum relative error of the function as R : $\exists R, \forall x$ we have $\frac{|f(x)-g(x)|}{|f(x)|} \leq R$.¹

Next, we assume that the two outputs of the LUT that enter the Check Layer are a and b (without loss of generality, we let $0 < a < b$). In the following, we need to derive the lower bound of R from a and b :

$$R \geq \max \left(\frac{|f(x) - a|}{|f(x)|}, \frac{|f(x) - b|}{|f(x)|} \right) \geq 0.5 \left(\frac{|f(x) - a|}{|f(x)|} + \frac{|f(x) - b|}{|f(x)|} \right) \geq 0.5 \left(\frac{|b - a|}{|f(x)|} \right). \quad (1)$$

Here, $f(x)$ is still present in Equation (1). As a result, we need to use a or b to replace the unobservable $f(x)$. Since:

$$\frac{|f(x) - a|}{|f(x)|} \leq R, \quad (2)$$

¹ Here we also require that $f(x)$ is not zero. For the definition of “non-zero,” see Definition 3.2. For a and b , which are mentioned later, we also require that they are not zero. In practice, when we encounter data which is not “non-zero” in the tensor, we skip it.

we have

$$\frac{|f(x)| - |a|}{|f(x)|} \leq R \Rightarrow |f(x)|(1 - R) \leq |a|. \quad (3)$$

There are two cases that follow. The first case is that R is not less than 1. This implies that the relative error exceeds 100%, indicating an anomalous precision of the operator. No further analysis is needed in this case. We focus on the scenario where the relative error is less than 1:

$$R \geq 0.5 \left(\frac{|b - a|}{|f(x)|} \right) \geq 0.5 \left(\frac{|b - a| \cdot (1 - R)}{|a|} \right). \quad (4)$$

Transforming Equation (4), we get:

$$2 \cdot |a| \cdot R \geq |b - a| \cdot (1 - R). \quad (5)$$

$$(2|a| + |b - a|) \cdot R \geq |b - a| \Rightarrow |a + b| \cdot R \geq |b - a|. \quad (6)$$

$$R \geq \frac{|b - a|}{|b + a|}. \quad (7)$$

This means that regardless of the specific function f being computed, we can obtain a lower bound on its maximum relative error using a large number of a and b values. Without any golden data, we can derive a lower bound: $R \geq \min \left(\frac{|b - a|}{|b + a|}, 1 \right)$. In subsequent experiments, we report the computational results of Equation (7). The reader should note that we assume the computational results of a and b have the same sign. This is generally natural, but for anomalous operators that miscalculate the signs of a and b , this may lead to an unusually large right-hand side in Equation (7).

With the measure approach of error in place, we can refine a series of testing criterion. But until then, the careful reader may notice that our approach may distort as $f(x)$ nears 0. As a result, let us start with the following definition.

3.1.1 Definition: Equivalence of Floating-point Numbers.

Definition 3.1 (Equivalence of Floating-point Numbers). We define the equivalence of floating-point numbers based on the precision of the machine's representation. Given two real numbers, A and B , and the minimum representation precision of the machine denoted by E , we consider A and B equal when $|A - B| \leq E$.

3.1.2 Definition: Non-zero.

Definition 3.2 (Non-zero). We define that a floating-point number A is non-zero if and only if $A \neq 0$.

Here, we are not directly comparing A with E to define non-zero. Instead, we want to allow for some margin, skipping small numbers altogether. For example, if the machine precision is $1e-8$, then when A is less than $1e-4$, we consider A to be very close to zero. If one of a , b is such a floating-point number, it will be disregarded for the relative error calculation. On the other hand, this has the advantage that if a and b have opposite signs, this means that the outputs, which should be the same, are off by a sufficiently large absolute value. For example, if the machine precision E is $1e-8$ and a and b have opposite signs, their absolute difference $|a - b|$ will exceed $(2e + 4) \cdot E$, indicating a significant discrepancy that should trigger an alert.

3.1.3 Theorem: Precision Criterion.

THEOREM 3.3 (PRECISION CRITERION). *Given a relative error threshold e , for any non-zero outputs a and b which should satisfy $a = b$, then the ratio $|a/b|$ is required to be between $(1 - e)/(1 + e)$ and $(1 + e)/(1 - e)$:*

$$\frac{1 - e}{1 + e} \leq \left| \frac{a}{b} \right| \leq \frac{1 + e}{1 - e}. \quad (8)$$

If there exists a set a, b that does not satisfy Equation (8), then the relative error R at this point must exceed the threshold e . According to this theorem, we can check whether the precision of the LUT exceeds the preset threshold e . The exact proof is similar to the previous derivation. The reader can obtain it by transforming Equation (7). The advantage of using such a criterion is obvious: It is free of false positives. Subject to ensuring that the inputs do not change, once an operator fails this test criterion, its relative error must exceed the threshold.

In this article, different e values are specified for various input types. Specifically, for `torch.bfloat16`, `torch.float16`, `torch.float32`, and `torch.float64`, e is set to $1e-2$, $1e-3$, $1e-4$, and $1e-8$, respectively.²

The error measurement approach outlined in this subsection serves as a robust foundation for our testing framework. The Check Layer, as further discussed in Theorem 3.3 Precision Criterion, diligently scrutinizes the relative errors between computed results and scientific calculations. It meticulously records instances where the computed results deviate beyond the defined threshold, ensuring a comprehensive evaluation of operator precision.

Now, as we transition to the next facet of our testing methodology, let us delve into the Input Perturbation in the Fuzz Layer. This pivotal component introduces perturbations to input tensors, facilitating the identification of layers and operators with low precision. By understanding how to construct perturbations through different strategies, we fortify the framework's versatility and applicability to diverse operator scenarios. Let us explore these concepts in the subsequent section.

3.2 Input Perturbation in Fuzz Layer

As evident from the preceding subsection, it is imperative to introduce a perturbation to the input $g^{-1}(a)$ (namely, A) to obtain $g^{-1}(b)$ (namely, B) in such a way that, for the machine/operator/algorithm, A and B are effectively identical.

We present two methods for constructing this perturbation. These two methods are applicable to arbitrary types of operators, while the subsequent subsection focuses on operators that satisfy specific properties.

3.2.1 Improve Precision. For different types of inputs, we enhance their precision by converting them to higher-precision types. Specifically, `torch.bfloat16` inputs are upgraded to `torch.float32`, `torch.float16` inputs are upgraded to `torch.float32`, and `torch.float32` inputs are upgraded to `torch.float64`. Inputs of type `torch.float64` are excluded from this process, as they are already at the highest precision supported.

Enhancing precision prevents values from overflowing while ensuring that A and B refer to the same real number. Consequently, the Precision Criterion (Theorem 3.3) helps eliminate false positives—that is, cases where g satisfies Theorem 3.3 but was incorrectly flagged as failing the precision test by FuMi.

3.2.2 Add Noise. Float is stored in binary in computers and each data type has a fixed number of decimal places. For example, `float16` has 10 binary decimal places. In other words, it represents

²The values of e are partially based on the reference in `torch.testing._internal.common_utils.dtype2prec_DONTUSE`, with some adjustments made. In practice, you can modify these values according to your specific needs.

roughly 4 to 5 decimal places accurately. So if we add a $1e-6$ noise to a float of type float16, the real number it represents remains practically unchanged.

We introduce perturbation by adding noise corresponding to the data type at all non-zero positions in a tensor. Specifically, for `torch.bfloat16`, `torch.float16`, `torch.float32`, and `torch.float64`, the added noise levels are $1e-4$, $1e-6$, $1e-8$, and $1e-16$, respectively. *Notably, noise is applied only to non-zero values.* For instance, at float32 precision, a noise level of $1e-8$ is added only if the absolute value of the data exceeds $1e-4$. This ensures that, in all cases, the relative perturbation remains below 0.01%.

3.3 Generalizability of FuMi

The two preceding subsections introduce a set of design concepts for a generalized Fuzz and Check. These tools are effective for most floating-point arithmetic. Moreover, our approach can be effortlessly extended to accommodate specific testing needs for particular operators.

3.3.1 Exchange Order. This perturbation is not universally applicable to the entire spectrum of float computational operators. It is specifically designed for operators whose mathematical properties adhere to commutativity. In the context of the input Tensor, perturbing by the specified dimension has no impact on the output. Such operators are frequently employed in distributed communication operations. For instance, the *all_reduce* operator can be conceptualized as summing up the results of computations from individual threads. Exchanging the results of computations between two threads does not affect the final sum. Consequently, this perturbation is suitable for detecting communication operators in multi-computer and multi-NPU scenarios.

3.3.2 Metamorphic Testing. Our framework can seamlessly integrate metamorphosis testing. For example, with the `sin` operator, we can construct the negative of A, denoted as $-A$, in the Fuzz Layer. Then, we can test $g(A)$, $-g(-A)$ in the Check Layer using the Precision Criterion from Theorem 3.3.

3.3.3 Custom Function. In DL, optimizing existing operators is a common practice. FuMi supports the measurement and testing of operator after optimization. In a subsequent experiment, we will demonstrate FuMi's capabilities using the `ATTENTION` operator from the popular transformer architecture as a case study. This experiment simulates the complete process of measuring the precision error of a developer's custom function against an existing function. As outlined in Section 2, the entire process is straightforward, requiring only the addition of a single parameter—the function name of the custom function.

For more (deterministic) custom layers, the principle of FuMi is as follows:

- There is a forward function in the layer.
- There is a fuzzable tensor argument in the layer's input.
- There is a checkable (i.e., float) tensor in the layer's output.

Then we can use FuMi to measure and test the precision of the layer.

To summarize, our approach to testing operator precision in DL frameworks using the Check Layer and Fuzz Layer is both robust and adaptable. However, it is important to emphasize that the primary focus of this article is on general rules that apply to most operators—specifically, that if the input remains unchanged, the output should also remain unchanged. As a result, we do not cover “exchange order” and “metamorphosis testing” in the following sections.³ Nevertheless, readers should recognize that our approach is highly flexible and can be easily extended.

³In other words, users must modify FuMi's code to incorporate “exchange order” and “metamorphosis testing.” For all other modes, FuMi can be used directly.

4 Evaluation

The following subsections detail the posed **research questions (RQs)** and the experimental setup.

4.1 RQs

- RQ1: Under the “improve precision” Fuzz Layer, how well does the operator maintain precision?

This question focuses on evaluating the precision maintenance of operators when subjected to the “improve precision” Fuzz Layer in the FuMi framework. The investigation seeks to understand the impact of precision enhancement mechanisms on the precision and reliability of operators within this specific fuzzing context.

- RQ2: Under the “add noise” Fuzz Layer, how well does the operator maintain precision?

This question focuses on evaluating the precision maintenance of operators when subjected to the “add noise” Fuzz Layer in FuMi, aiming to assess how well operators preserve precision when exposed to this particular fuzzing condition. Examining precision maintenance under different fuzzing strategies provides insights into the robustness and adaptability of operators in the presence of noise.

- RQ3: How efficient is FuMi?

This question focuses on evaluating the precision maintenance of operators when subjected to the “improve precision” Fuzz Layer in FuMi. This question explores the overall efficiency of the FuMi framework. It delves into the effectiveness and performance of FuMi in carrying out its intended tasks, considering factors such as speed, resource utilization, and overall computational efficiency. This question provides a holistic perspective on the practicality and utility of FuMi as a fuzzing tool.

- RQ4: Comparison between FuMi and Predoo algorithm by operator ATTENTION—Which can show the larger relative error?

This question focuses on evaluating the precision maintenance of operators when subjected to the “improve precision” Fuzz Layer in FuMi. The goal of this section is to compare the accuracy of FuMi and Predoo algorithm in measuring the relative error of the ATTENTION operator. Since both tools measure the lower bound of the relative error, the tool that reports a larger relative error is considered to be more accurate. By comparing the results, we aim to determine which tool provides a more precise evaluation of the operator’s performance. This comparison will help highlight the strengths of each tool in identifying and quantifying errors within DL models.

4.2 Experimental Setup

4.2.1 Hardware and Software. The experiments were conducted on a Linux system equipped with a Intel(R) Xeon(R) Gold 6132 CPU @ 2.60 GHz, 772 G RAM, and a Tesla V100-SXM2-16 GB GPU [26]. The system ran on CUDA version V12.1.66, and Python version 3.8.19.

However, it is crucial to note that the methodology proposed in this article is inherently general and not restricted to a specific computing chip. In fact, by computing the relative error across multiple chips, a comprehensive comparison of computational precision can be conducted. Unfortunately, due to cost considerations and privacy policies, the exploration of this RQ will be addressed in future work.

4.2.2 Benchmark. Nine models from PyTorch’s examples [24] were selected for investigation, spanning various machine learning tasks such as CV, **natural language processing (NLP)**, and **reinforcement learning (RL)**. The chosen models include AlexNet [19], ResNet18 [12], ResNet50 [13], LSTM [14], RNN [10], CNN, RL [25], transformer [27], and Auto-Encoding Variational Bayes [18]. For LSTM and RNN, we utilized the implementation provided in PyTorch’s

Table 1. Number of Operator Calls

Operator/Model	AlexNet	ResNet18	ResNet50	LSTM	RNN	LeNet	DQN	Transformer	Variational Bayes
conv2d	156,030	312,166	2,204,800	-	-	26,544	-	-	-
batch_norm	-	312,166	2,204,800	-	13,272	-	-	-	-
binary_cross_entropy	-	-	-	-	-	-	-	-	5,480
adaptive_avg_pool2d	31,206	15,608	41,600	-	-	-	-	-	-
cross_entropy	31,206	15,608	41,600	-	-	-	-	-	-
embedding	-	-	-	22,338	-	-	-	3,606	-
layer_norm	-	-	-	-	-	-	-	18,030	-
linear	93,618	15,608	41,600	22,338	26,544	26,544	100,652	32,454	27,420
log_softmax	-	-	-	22,338	13,272	13,272	-	3,606	-
max_pool2d	93,618	15,609	41,600	-	-	-	-	-	-
nll_loss	-	-	-	22,338	13,272	13,272	-	3,606	-
relu	218,442	265,341	2,038,400	-	13,272	39,816	50,326	-	10,970
softmax	-	-	-	-	-	-	50,326	-	-
abs	-	-	-	-	-	-	50,326	-	-
exp	-	-	-	-	-	-	-	-	5,480
mean	-	-	-	-	-	-	365	-	-
mul	564,102	282,051	7,752,136	-	-	-	-	-	20
sqrt	-	-	-	-	-	-	1,460	-	-
std	-	-	-	-	-	-	365	-	-
sum	62,412	31,216	83,200	-	-	-	101,017	-	-
add	-	-	-	-	262,640	-	-	-	-

word_language_model example. This model is a classic language modeling framework based on LSTMs and RNNs, which have been widely used in NLP tasks. For CNN, we tested the CNN-based models from the mnist example. These include classical architectures like LeNet, covering both shallow and deep CNN variations. These models were selected for their historical significance and widespread application in CV tasks. For RL, we tested the **deep Q-network (DQN)** implementation provided in PyTorch’s reinforcement_learning example. For transformer, we used the implementation from PyTorch’s translation example, which is based on the original “Attention Is All You Need [27]” architecture proposed by Vaswani et al. This choice was made because of its widespread adoption and proven effectiveness in NLP tasks.

We conducted an investigation of all the APIs involved in numerical computation in PyTorch across nine models. Besides this, we aimed to simulate the process of measuring the precision of user-defined custom functions. For our research, we selected the LLaMA model. Readers can find specific calls to the ATTENTION operator in the modeling_llama.py file, which is part of the dependency package installed via “pip install transformer.” We modeled the process of replacing the default calls with alternative attention algorithms and measured the relative errors involved.

This approach allowed us to evaluate the precision and robustness of different attention mechanisms within the LLaMA model, ensuring that our custom implementations meet the required precision standards.

4.2.3 LUT. We evaluated 167 APIs related to floating-point computation from the torch.Tensor and torch.nn.functional modules, treating each specific operator as a LUT. For these operators, a total of 17,982,324 tests were conducted. However, it is important to note that not all 167 APIs listed as testable were tested in our experiments; for more detailed information, please refer to Table 1. Additionally, we used the ATTENTION operator in transformers as a case study for custom functions. While newer technologies such as flash_attention have been introduced by the official PyTorch package, our experiments emphasize the flexibility of FuMi when applied to user-defined functions.

By thoroughly testing these APIs and incorporating custom function evaluations, we demonstrate FuMi's capability to adapt to evolving technologies and maintain robust error measurement and optimization processes. This approach not only underscores FuMi's versatility but also ensures that user-defined functions meet precise accuracy standards within DL frameworks.

4.2.4 Baseline. This article presents the inaugural test framework explicitly designed for runtime precision testing. As a baseline, we utilize Predoo algorithm [32], a unit test tool tailored for operators. The comparison methodology involves each tool identifying the test cases that most effectively reveal defects. Subsequently, we assess the calculations of these test cases against golden data, as detailed in RQ4. It is crucial to highlight that Predoo algorithm requires manual test code creation for individual operators (further, this article's authors adapted Predoo tool from TensorFlow to PyTorch) and lacks the generalizability to the entire spectrum of operators, a feature inherent in FuMi. Therefore, our comparison is conducted specifically on the operator ATTENTION featured in this article. Besides Predoo, we also investigated arithmetic testing tools including FreeFuzz [29], VFuzz [30], and DeepREL [8]. For the comparison and discussion, see Section 6.1.

5 Results

5.1 RQ1: Precision Issues Found by Improving Precision

We aggregated a total of 59 operators from nine models into the Fuzz Layer (i.e., 59 data points can be seen in Table 1), encompassing 21 different operators. Table 1 displays the number of calls for each operator. Notably, many operators are invoked tens of thousands of times during model training, making it valuable to assess the worst precision performance for researchers to address precision issues in model training. Among the numerous invocations, we subjected them to precision testing in the Check Layer based on Theorem 3.3 Precision Criterion. Different ϵ values are specified for various input types. Specifically, for torch.bfloat16, torch.float16, torch.float32, and torch.float64, ϵ is set to $1e-2$, $1e-3$, $1e-4$, and $1e-8$, respectively.

Table 2 provides statistics on the percentage of error larger than threshold and reports the lower bound of relative error observed. We report the largest relative error that we measure. Clearly, the measured relative error represents a lower bound of the maximum relative error across the entire input space. If the measured lower bound exceeds the threshold defined by our precision criterion, the operator is considered to have failed to meet the criterion. For such an operator, we further report the proportion of calls where the relative error exceeds the given threshold. This proportion is presented in parentheses. Bold formatting is applied to all data where the percentage is not 0. Due to variations in output precision, some operators are labeled as substandard, even if the lower bound is less than 0.01. For instance, `linear`, despite having relative error less than 0.01, is deemed substandard due to its higher output precision, prompting a more rigorous criterion.

Analysis of the table reveals that the five operators with the most significant precision challenges are `conv2d`, `cross_entropy`, `log_softmax`, `batch_norm`, and `linear`. These operators exhibit substantial errors in computation results and occur more frequently. For `conv2d` and `cross_entropy`, there will also be cases where the sign of the outputs is reversed by inputs with different dtypes. For `conv2d`, besides the problem of sign, the relative error can be more than 0.5%. This underscores a warning to model trainers, urging them to exercise caution in balancing precision and efficiency when employing these operators.

5.2 RQ2: Precision Issues Found by Adding Noise

Table 3 presents similar statistics, emphasizing the percentage of cases where the error does not meet the threshold, accompanied by the lower bound of the relative error. Bold formatting is used for data points with non-zero error percentages. As with Table 2, operators like `linear` may still be

Table 2. (RQ1) Lower Bound of Relative Error and Percentage of Error Larger than Threshold

Operator/Model	AlexNet	ResNet18	ResNet50	LSTM	RNN	LeNet	DQN	Transformer	Variational Bayes
conv2d	25.689 (29.6%)	2.159 (100%)	81.359 (62.3%)	-	-	0.00519 (100%)	-	-	-
batch_norm	-	0.004 (89.1%)	0.00897 (34.2%)	-	0.01111 (87.2%)	-	-	-	-
binary_cross_entropy	-	-	-	-	-	-	-	-	7e-8(0)
adaptive_avg_pool2d	0(0)	2e-07(0)	0(0)	-	-	-	-	-	-
cross_entropy	2e-07(0)	2e-07(0)	1.8(0.1%)	-	-	-	-	-	-
embedding	-	-	-	0(0)	-	-	-	0(0)	-
layer_norm	-	-	-	-	-	-	-	0.00030(17.2%)	-
linear	0.00023 (0.1%)	0.00167 (6.3%)	0.00061 (0.1%)	0.01094 (100%)	0.00138 (5.1%)	0.00198 (16.3%)	0.00061 (0.03%)	0.00455 (99.1%)	0.00350 (60.2%)
log_softmax	-	-	-	0.00371 (66.4%)	0.00135 (62.1%)	0.00101 (94.5%)	-	0.00035 (0.9%)	-
max_pool2d	0(0)	0(0)	0(0)	-	-	0(0)	-	-	-
nll_loss	-	-	-	2e-7(0)	2e-7(0)	4e-7(0)	-	0(0)	-
relu	0(0)	0(0)	0(0)	-	0(0)	0(0)	0(0)	-	0(0)
softmax	-	-	-	-	-	-	3e-7(0)	-	-
abs	-	-	-	-	-	-	0(0)	-	-
exp	-	-	-	-	-	-	-	-	7e-8(0)
mean	-	-	-	-	-	-	1e-7(0)	-	-
mul	0(0)	0(0)	0(0)	-	-	-	-	-	3e-8(0)
sqrt	-	-	-	-	-	-	3e-8(0)	-	-
std	-	-	-	-	-	-	3e-8(0)	-	-
sum	0(0)	0(0)	0(0)	-	-	-	4e-5(0)	-	-
add	-	-	-	-	2e-8(0)	-	-	-	-

Explanation. The table presents the lower bound of the relative error computed by Equation (7), along with the percentage of error that exceeds the threshold. Bold formatting is used to highlight data where the percentage of error is non-zero. When the relative error exceeds 1, this indicates that the operator produces results where the operands a and b have opposite signs. Meanwhile, as noted in Section 3, we guarantee that both a and b are at least 100 machine precision away from zero to avoid rounding errors.

Table 3. (RQ2) Lower Bound of Relative Error and Percentage of Error Larger than Threshold

Operator/Model	AlexNet	ResNet18	ResNet50	LSTM	RNN	LeNet	DQN	Transformer	Variational Bayes
conv2d	586.94 (25.8%)	0.08564 (100%)	0.19399 (66.3%)	-	-	0.00452 (66.3%)	-	-	-
batch_norm	-	0.00268 (90.4%)	0.00395 (61.6%)	-	0.00501 (40.5%)	-	-	-	-
binary_cross_entropy	-	-	-	-	-	-	-	-	1e-7(0)
adaptive_avg_pool2d	5e-5(0)	1e-05(0)	3e-5(0)	-	-	-	-	-	-
cross_entropy	1e-07(0)	7e-08(0)	7e-8(0)	-	-	-	-	-	-
embedding	-	-	-	5e-7(0)	-	-	-	5e-5(0)	-
layer_norm	-	-	-	-	-	-	-	0.00028 (4.4%)	-
linear	0.00015 (0.1%)	0.00038 (13.8%)	0.00379 (19.3%)	0.00235 (100%)	0.00029 (0.2%)	0.00128 (5.7%)	0.00105 (0.02%)	0.00145 (80.9%)	0.00224 (21.1%)
log_softmax	-	-	-	3e-5(0)	1e-5(0)	1e-5(0)	-	1e-6(0)	-
max_pool2d	5e-5(0)	5e-5(0)	5e-5(0)	-	-	5e-5(0)	-	-	-
nll_loss	-	-	-	0(0)	0(0)	0(0)	-	0(0)	-
relu	5e-5(0)	5e-5(0)	5e-5(0)	-	5e-5(0)	5e-5(0)	0(0)	-	5e-5(0)
softmax	-	-	-	-	-	-	9e-8(0)	-	-
abs	-	-	-	-	-	-	0(0)	-	-
exp	-	-	-	-	-	-	-	-	6e-8(0)
mean	-	-	-	-	-	-	0(0)	-	-
mul	0(0)	0(0)	0(0)	-	-	-	-	-	5e-5(0)
sqrt	-	-	-	-	-	-	3e-5(0)	-	-
std	-	-	-	-	-	-	0(0)	-	-
sum	0(0)	0(0)	0(0)	-	-	-	2e-5(0)	-	-
add	-	-	-	-	5e-5(0)	-	-	-	-

marked substandard despite a lower bound of less than 0.01, owing to the need for more stringent precision in their computations.

Analysis of the table reveals that the four operators with the most significant precision challenges are conv2d, batch_norm, layer_norm, and linear. These operators exhibit substantial errors in computation results and occur more frequently. This underscores a warning to model trainers, urging them to exercise caution in balancing accuracy and efficiency when employing these operators.

Upon comparing Tables 2 and 3 for the two different modes, it becomes evident that the problematic operators on the V100 exhibit a significant overlap. However, it is crucial to note that this overlap does not imply similarity in the types of defects addressed by the two modes. We have identified bugs on other chips⁴ that are exclusively exposed in the “add noise” mode. Furthermore, there are instances where “improve precision” might not be applicable. This could be due to the utilization of the highest precision or a specific operator not supporting higher precision or out of

⁴The author worked for a well-known chip manufacturer and is unable to disclose relevant chip data due to privacy policies.

Table 4. Training Time for Each Model

Time/Model	AlexNet	ResNet18	ResNet50	LSTM	RNN	LeNet	DQN	Transformer	Variational Bayes
Original	262 min 1 s	259 min 59 s	108 min 23 s	2 min 29 s	4 min 6 s	3 min 32 s	0 min 34 s	2 min 14 s	1 min 13 s
Add Noise	1,086 min 3 s	3,088 min 52 s	1,064 min 24 s	40 min 11 s	6 min 35 s	8 min 21 s	1 min 51 s	7 min 13 s	2 min 52 s
Improve Precision	603 min 40 s	847 min 10 s	880 min 31 s	9 min 20 s	7 min 6 s	5 min 58 s	3 min 6 s	4 min 17 s	1 min 45 s
Add Noise TPC ^a	39 ms	134 ms	4 ms	25 ms	0.4 ms	2 ms	0.2 ms	21 ms	2 ms
Improve Precision TPC	16 ms	16 ms	3 ms	5 ms	0.5 ms	1 ms	0.4 ms	9 ms	0.6 ms

^aTPC records the average time per check. For example, AlexNet generated 1,250,634 operator calls (sum of first column in Table 1). The additional elapsed time with “add noise” is about 824min and 2s. So the average elapsed time per call is about 39ms.

TPC, Time-per-check.

memory. As a result, on the other hand, the “add noise” mode is applicable in a broader range of scenarios. We delve into the types of defects targeted by the two different modes in discussion.

5.3 RQ3: Efficiency

In our experiments, the additional overhead in FuMi stems from two primary sources.

Computational Overhead of Fuzz Layer, LUT, and Check Layer. During the check phase, `torch.where` and `torch.div` need to be executed once. The overhead of fuzz and LUT depends on the mode. In the add noise mode, extra resources are required to execute `torch.where`, `torch.ones`, and `torch.add` once during the perturbation phase. Additionally, the LUT operator needs to be repeated once with the same complexity. In the improve precision mode, two type conversions are needed, and the LUT operator has to repeat an operation of higher complexity. The computational overhead of doubling the precision may exceed a simple doubling. For instance, multiplying a two-digit number by another two-digit number not only involves four times the time overhead of one-digit * one digit for multiplication but also additional overhead for “add.” Table 4 reports the execution time of each model under original settings and the time when using FuMi’s different runtime precision-checking modes (Add Noise and Improve Precision), including time-per-check metrics.

Overhead in Recording Operator Calls and Storing Abnormal Inputs and Outputs. This refers to the number of times we count the calls to the full operator when needed for the experiment. A more time-efficient approach is not to count the number of calls but to store only the inputs and outputs that do not meet the Precision Criteria. In our real test scenarios, we generally opt for this option, reducing the overhead of repeatedly reading and writing the statistics file and making notes.

Additionally, FuMi is constrained by the number of test subjects. In our experimental setup, we pre-set 167 APIs as subjects to be tested. While not all APIs will be involved in the model, we avoid nested calls, such as “add” and “mul” in linear operators. However, these calls are still needed to check the “nested,” introducing a time overhead.

Let us delve into the average time required to test each operator. For example, AlexNet generated 1,250,634 operator calls (sum of first column in Table 1). a single invocation and assessment of an operator take no more than 39ms in the “add noise” mode and 16ms in the “improve precision” mode, on average. Across all models, FuMi consistently maintains a test response time of less than 50 ms (except adding noise for ResNet18), showcasing a high level of efficiency.

In summary, while FuMi introduces some computational overhead, particularly in the form of additional execution steps during the perturbation and check phases, it proves to be an efficient testing framework for assessing operator precision. The choice between add noise and improve precision modes influences the computational demands, with the latter generally requiring more time due to the complexity of operations. The option to selectively record and store abnormal inputs and outputs helps streamline the process. Despite these considerations, FuMi demonstrates effectiveness in testing a diverse set of APIs, offering valuable insights into operator precision.

Table 5. Tensor Statistical Data

ATTENTION	Shape	Max	Min	Mean
Predoo	(1 * 3 * 10 * 10)	1.3600057652	-1.5673176257	0.0022148285
FuMi	(20 * 2 * 35 * 100)	2.7771960364	-2.8309485647	-0.0113029344

Table 6. The Lower Bound of Relative Error for “ATTENTION” Computed by Different Tools

FuMi	Predoo
1.04%	0.08%

As we continue refining and optimizing the framework, the aim is to strike a balance between precision assessment and computational efficiency, ensuring a robust testing tool for DL practitioners.

5.4 RQ4: Comparison to Predoo Algorithm

In this section, we compare the process and results of measuring the error of the ATTENTION operator using Predoo algorithm and FuMi. Our evaluation criterion is straightforward: Since both tools assess the relative error of the ATTENTION operator, each tool provides a lower bound on this error based on its observations. The tool that reports a larger lower bound on the relative error is deemed more accurate, as it is closer to the true global lower bound of the error. To ensure fairness, both tools are tested with 20,000 groups of inputs.

For FuMi, we simply need to import the package and decorate the ATTENTION function with the @decorator function. During a single iteration of the actual LLaMA run, we observed a total of 330,480 calls to the ATTENTION operator. Among these calls, FuMi calculated a lower bound on the relative error of 1.47%. Specifically, for the first 20,000 calls, the observed lower bound was 1.04%.

In contrast, using Predoo algorithm requires substantial effort. First, we had to convert the original Predoo code from TensorFlow to PyTorch. Additionally, we needed to understand the input requirements for the ATTENTION operator: Q, K, and V must have the same shape. Finally, given the vast input space, exhaustive testing is impractical, so we followed Predoo’s original approach by testing with a specified shape. Specifically, we used the shape [1, 3, 10] for unit testing, as done by the original Predoo tool. After 20,000 calls, Predoo reported the outputs (which should be the same) with the largest differences being 1.6689453125 and 1.6662929037. However, readers must note that the Predoo algorithm, we cannot get the estimation of error since we cannot know the ground truth. As a result, we can only compute the lower bound by Equation (7). By Equation (7), we can determine that the lower bound of the relative error is 0.08%, which is less than one-tenth of FuMi’s result. This indicates that significant additional work is required to use the Predoo unit testing tool, and even after all this work, the unit tests’ measurements of the arithmetic accuracy can be grossly underestimated due to the variability between the real inputs and the true inputs. From Table 5 we can recognize the difference between real and simulated data. From Table 6 we can compare the lower bound of relative error computed by FuMi against Predoo.

It is important to highlight that Predoo algorithm, although it also appears to switch between different precisions, is fundamentally different from FuMi. Predoo algorithm switches between any two precisions, which can lead to a loss of precision: When switching from high precision to low precision, the switch does not necessarily point to the same real number. For instance, truncating

```
>>> r2.flatten()[:10]
tensor([ 3.8214206696e-02, -1.3462867737e+00,  3.5038616180e+01,
        3.5608352661e+01, -7.0018844604e+00,  1.1234144211e+01,
        5.4831898212e-01,  9.8258972168e-02,  2.2060546875e+00,
        4.6680480957e+01], device='cuda:0', grad_fn=<SliceBackward0>)
>>> r1.flatten()[:10]
tensor([ 3.8090442289e-02, -1.3464928944e+00,  3.5038391769e+01,
        3.5608058653e+01, -7.0019396676e+00,  1.1234121791e+01,
        5.4830942101e-01,  9.7962474410e-02,  2.2059055350e+00,
        4.6680326733e+01], device='cuda:0', dtype=torch.float64,
        grad_fn=<SliceBackward0>)
```

(a) Comparison of normal data

```
>>> index
tensor(15631, device='cuda:0')
>>> r2.flatten()[index]
tensor(-0.0002822876, device='cuda:0', grad_fn=<SelectBackward0>)
>>> r1.flatten()[index]
tensor(0.0012790829, device='cuda:0', dtype=torch.float64,
        grad_fn=<SelectBackward0>)
```

(b) Comparison of anomalous data

Fig. 3. Anomalous data exposed by FuMi.

an 8-bit decimal to a 4-bit decimal changes the inputs themselves. Mathematically, the resulting outputs should not be equal. Therefore, the results reported by Predoo may also be questionable (i.e., false positives) due to methodological imperfections. In contrast, FuMi only performs a precision boost to ensure that a low-precision value is representable at high precision, theoretically pointing to the same real number.

These observations highlight two significant advantages of FuMi. Firstly, FuMi is simpler to use, as it does not require independent unit tests for each operator, unlike Predoo algorithm. This simplicity enhances FuMi's automation capabilities. Secondly, FuMi provides a larger lower bound on the relative error based on real input, suggesting a more accurate assessment of operator precision compared to Predoo.

6 Discussion

This section primarily focuses on three key areas: a case study for the conv2d operator, an analysis of false negatives, and the ease of FuMi. The first subsection provides an in-depth examination of how the conv2d operator behaves with different signs. The second subsection explores the types of defects that FuMi can and cannot detect, with a detailed discussion on the conditions leading to false negatives. The third subsection demonstrates FuMi's practical usability through its automated testing workflow and developer-friendly design.

6.1 Case Study for conv2d

In this section, we will analyze and compare five approaches: FuMi, Predoo, FreeFuzz, ∇ Fuzz, and DeepREL. We will provide a detailed analysis of the results for the conv2d operator, specifically focusing on the ResNet50 model for FuMi. We begin by examining a case where the computation results in outputs with opposite signs. Both the original and fuzzed outputs were saved for comparison.

Let us start by looking at the results of the max error caught by FuMi in the first 20,000 calls. Figure 3 presents the first 10 floating-point values from both output tensors. As shown, they are approximately the same, with the first four decimal places being identical. This indicates that the two outputs should be very close to each other. However, anomalies become evident when we examine the location of the maximum error, as depicted in Figure 3.

Table 7. Maximal Precision Discrepancy

conv2d	Ground Truth (CPU)	Original Output	Fuzzed Output	Relative Error	Absolute Error
Predoo	2.099609375	2.101562500	2.0993984830	0.093%	0.0019
FuMi	0.0012790821	-0.0002822876	0.0012790829	122.1%	0.0016
FreeFuzz	5.212492943	5.212492943	5.212495327	2.28e-5%	2.38e-6

A significant difference (0.0013 vs. -0.0003) is observed at the position with the largest relative error (index = 15,631), where the values have opposite signs. It is important to note that at float64 precision, 0.0013 is not an insignificant number. In this tensor, out of 802,816 values, 3,862 have an absolute value less than 0.0013, which constitutes about 0.48% of the total.

We then sent the input to the CPU for computation. The result under float64 is 0.0012790829 and under float32 is 0.0012790821. It can be seen that there is a significant error in the results of the calculations using CUDA under float32 in this network. The absolute error is 0.0016 and the relative error is 122.1%.

Next, we conducted a unit test using Predoo algorithm. After 20,000 iterations, Predoo found the data with the largest difference by first converting the input from float64 to other types. After calculating the error under different types, the maximum error found was that the output was 2.0993984830 under float64 and 2.1015625000 under float16. However, we must note that the process of converting the inputs from float64 to float16 introduces a loss of precision. Therefore, to define the error accurately, we should use the results of the calculation on the CPU as a reference. On the CPU, the output is 2.0993984830 (exactly the same) for float64 and 2.0996093750 for float16. This indicates that the mathematical result should not be the same due to the precision loss from high precision to low precision conversion. Therefore, using Predoo will overestimate the error in this example. The actual absolute error is 0.0019 and the relative error is 0.093%.

Table 7 summarizes the results of this comparison, so that the reader can see at a glance which tool is better at exposing the error of the operator. We have included the full example in the code repository for the reader to reproduce. Note, however, that a V100 GPU is required to reproduce these results accurately, and different results might be obtained if other GPUs are used.

Next, we investigated the capabilities of FreeFuzz for unit testing on the conv2d operator. However, after generating 9,839 test cases for conv2d by FreeFuzz (available in our repository), we unfortunately found that 8,521 of them resulted in illegal inputs for conv2d. For the remaining 1,318 test cases, we report the results with the largest relative error among them. The actual absolute error is 2.38e-6 and the relative error is 2.28e-5%. This result shows that FreeFuzz is slightly weaker than Predoo.

After that, we thoroughly analyzed the 86.6% (i.e., 8,521/9,839) failed test cases and FreeFuzz's methodology. We discovered that FreeFuzz generates inputs based on type and samples random numbers for mutated shapes and values. This approach faces challenges when inputs must satisfy specific constraints. For instance, in test case 752:

```

. . .
arg_1_tensor = torch.rand([80, 64, 30, 30], dtype=torch.float32)
arg_1 = arg_1_tensor.clone()
arg_2_tensor = torch.rand([64, 64, 1, 7], dtype=torch.float32)
. . .

```

This input produces the following error:

Table 8. Comparison of Testing Tools

Tool	Testing Node	Requires Dump	Crash Testing Capability	Precision Testing Capability	Depends on Ground Truth	Produces False Positives
FuMi	Runtime Testing	No	Weak	Strong	No	No
Predoo	Unit Testing	No	Weak	Strong	Yes	Yes
Dump Tools	Runtime Testing	Yes	Weak	Strong	Yes	No
FreeFuzz	Unit Testing	Yes	Strong	Moderate	Yes	No
DeepREL	Unit Testing	Yes	Strong	Moderate	Yes	Yes

RuntimeError: Given groups=30, expected weight to be divisible by 30 at dimension 0, but got weight of size [[64, 64, 1, 7]] instead

It shows that randomly generated data is almost impossible to satisfy specific constraints. As a result, we believe such comparisons may not be entirely fair to FreeFuzz because it demonstrates its strength in generating illegal inputs that expose crashes. This is a valuable capability, even if its ability on precision testing might be comparatively weaker in certain cases. Similarly, DeepREL faces the same issue. Meanwhile for ∇ Fuzz, the test cannot be performed in the forward function. As a result, it is impossible to test the computational precision of the conv2d.

We provide comparison in Table 8 to summarize the strengths and weaknesses of various methods. This table is intended to help practitioners select the most suitable testing tools and techniques based on their specific needs.

6.2 Analysis of False Negatives

As demonstrated in the analysis presented in Section 3, FuMi exhibits a lack of false positive, assuming the validity of the underlying assumptions. In this Discussion section, we delve into the defect types detectable by different perturbation modes and examine the conditions under which false negatives may arise.

For “add noise” modes involving the addition of noise, they essentially serve as robust tests for the continuity of the function. While machine-computed floating-point functions must inherently be intermittent and discrete, significant shifts in function values when using the two most neighboring floating-point numbers as inputs may indicate a substantial deviation from the results of scientific computation. This scenario is abstracted as a form of outlier (see Figure 4(a)). Of course some readers may wonder if increasing the noise will still ensure that the input is constant. In fact, this article uses an extremely conservative approach to increasing this noise, and the reader can review the results reported in the experiments. “Adding noise” in the vast majority of cases, the lower bound of the measured relative error is less than the relative error under “improving precision.” This side by side confirms that adding noise can achieve no false positives under the two premises of (1) noise that is slightly less than the machine’s accuracy; and (2) adding noise only to data that is large enough in absolute value (to ensure that the relative change is extremely small).

In the case of “improve precision” mode, this mode effectively functions as a consistency test for various implementations of the function. Many operators run different code implementations for high precision inputs compared to low precision inputs. A substandard implementation reported in “improve precision” mode suggests a considerable offset in the computational results between the two implementations. This situation is abstracted as a poor fit of $g(x)$ to $f(x)$ obtained by a particular realization (see Figure 4(b)).

In instances where FuMi fails to detect the defect type, certain theoretical conditions should be met: (1) the continuity is good, (2) the difference between high-precision and low-precision realizations is negligible, and (3) errors with scientific computation results are out of the tolerable error range (see Figure 4(c)).

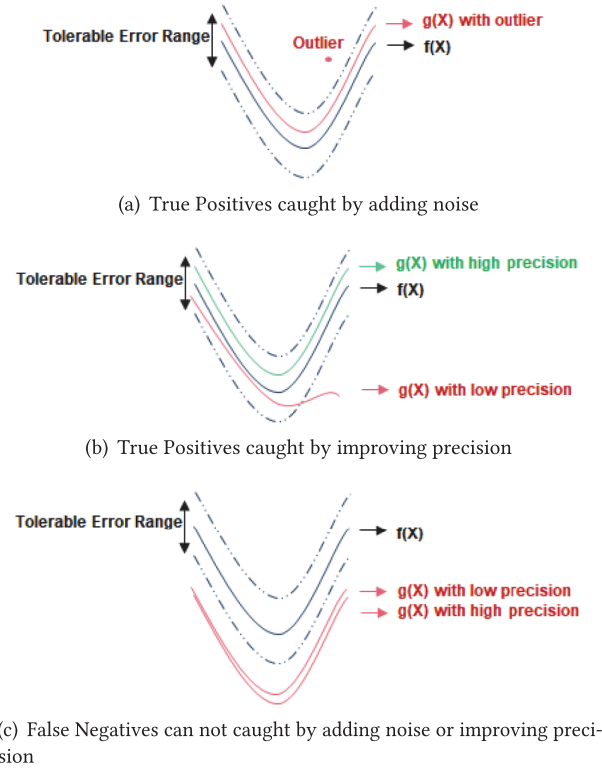


Fig. 4. The diagram abstracts the types of defects that FuMi can detect and the types of defects that it cannot detect.

6.3 Ease of Testing That FuMi Offers

The FuMi framework is designed to streamline the process of testing precision in DL operators, providing a highly practical and accessible approach for researchers and practitioners alike. Unlike traditional testing methodologies that often rely heavily on golden data or manual error detection, FuMi offers a more automated and efficient solution, reducing the testing burden while maintaining high accuracy in identifying precision issues.

No Dependence on Golden Data. One of the key advantages of FuMi is its ability to operate without the need for golden data, which is typically a significant limitation in existing testing methodologies. Golden data, while essential for comparison, can be difficult to obtain, especially in DL tasks where the inputs and outputs are highly complex. FuMi bypasses this requirement by utilizing Fuzz and Check Layers, which allow for precision evaluation during runtime without relying on a pre-defined “ground truth” dataset. This approach makes the testing process more practical, especially for large-scale DL models, and is a major step forward in advancing testing methodologies in the field of DL.

Practical Implementation for Developers. FuMi provides an easy-to-implement solution for developers working with DL frameworks. The Pythonic design of the framework allows for easy integration into existing workflows, with minimal setup required. Developers can incorporate Fuzz and Check Layers into their testing routines to automatically detect precision issues during model training. This offers a powerful tool for identifying potential errors early in the development process, leading to more robust and accurate models.

Scalability across Operators. FuMi is designed to be scalable across a variety of operators and model architectures, making it versatile for different use cases. Whether the operator is a simple layer function like addition or a more complex operation such as ATTENTION, FuMi can seamlessly evaluate precision and uncover discrepancies between the theoretical and actual computational results. This makes it an ideal tool for both small-scale experiments and large production systems, where testing all potential operator issues manually would be impractical.

In summary, FuMi simplifies the process of precision testing for DL operators, offering an efficient, golden-data-independent approach that can be easily adopted by both researchers and developers. By automating error detection and providing scalable solutions, FuMi significantly enhances the reliability and precision of DL models.

7 Related Work

Fuzzing techniques in the realm of DL have witnessed diverse applications and methodologies. Early efforts, exemplified by works like Guo et al. [11], Hu et al. [15], and Wang et al. [28], primarily concentrated on generating adversarial examples to evaluate the robustness of DL models. However, this article takes a departure from the conventional focus by addressing a unique challenge within the domain.

In contrast to the adversarial-centric approaches, our primary goal is to harness fuzzing techniques for identifying layers and operators exhibiting low precision when dealing with extensive datasets and large DL models. This distinctive focus sets our work apart from the aforementioned strategies, marking a novel contribution to the field. *At the same time, previous Fuzz work is difficult to use as the baselines because they cannot produce an estimate of precision.*

Recent work more closely related to our study in operator testing includes both crash-focused fuzz testing frameworks (FreeFuzz [29], ∇ Fuzz [30], and DeepREL [8]) and precision-focused approaches (Zhang et al. [31, 32] and Liu et al. [21]). ∇ Fuzz specializes in differential fuzz testing for DL but suffers from limited applicability due to its reliance on automatic differentiation. FreeFuzz and DeepREL automatically construct test inputs by analyzing popular models to extract operator signatures and input patterns, though they primarily detect crashes rather than precision issues due to their inability to automatically infer input constraints. Similarly, Predoo of Zhang et al. and work of Liu et al. employ generated inputs for precision testing, but rely on simulated test cases rather than real training data.

Our approach differs fundamentally in three aspects: (1) we use actual runtime training data instead of simulated inputs, (2) we employ relative error metrics with parameterized thresholds rather than absolute error measures, and (3) we establish pre-defined precision criteria before testing. For example, knowing that conv2d's high-low precision difference is $3.56e-3$ is meaningless without context—our parameterized thresholds provide this essential context. While existing approaches such as Predoo [32] rely on empirically chosen test parameters provided by TensorFlow developers, our method enables systematic precision evaluation across a broader range of operators.

8 Threats to Validity

Threats to validity arise from two primary sources, the first of which pertains to the complete validity of our hypotheses. The second aspect involves whether the API used in our FuMi tool introduces precision issues.

For the first threat, in other words, should “if the input stays the same, then the output should stay the same mathematically” be hold? In theory, this should need to be guaranteed. But from our point of view, it might be possible to be a little more lenient with our computers: For an operator that is implemented correctly and with up-to-date precision, if the INPUT doesn't change (the change is less than the machine precision), there exists a constant k such that the change in the OUTPUT is

less than k times the machine precision. We call such an operator a k -Lipschitz continuous operator [9]. For k -Lipschitz continuous operators, the test criterion should be modified to test criterion II.

THEOREM 8.1 (PRECISION CRITERION II). *Given a relative error threshold e , for any non-zero outputs a and b computed by a k -Lipschitz continuous operator which should satisfy $|a - b| < k * E$, then the following should hold:*

$$\frac{|a|}{1+e} \leq \frac{|b|}{1-e} + k * E, \text{ and } \frac{|b|}{1+e} - k * E \leq \frac{|a|}{1-e}. \quad (9)$$

So to address the first threat, we propose Precision Criterion II. This article can be seen as a special case of $k = 0$. Relying on this approach it is also possible to categorize the operators into 0-Lipschitz continuous operator, 1-Lipschitz continuous operator, and so on. We plan to do some expansion of this in the future.

The second threat stems from the torch operators utilized in the FuMi code. For instance, in the case of division calculations, if the div operator provided by torch lacks satisfactory precision, it might impact the final conclusion. While the ideal solution would involve using numpy's div operator, this introduces significant overhead: encompassing tensor conversion to numpy arrays and subsequent computation. This impracticality renders numpy unsuitable. To mitigate this threat, we conducted a random sampling of 10,000 pairs of float16 floating-point divisor' inputs and assessed the differences between numpy and PyTorch. The results obtained ensure a maximum relative error of less than $1e-6$.

9 Implications

Our study has important implications for both researchers and practitioners. The detailed implications are listed as follows:

- *Advancement in Testing Methodologies.* Our proposed testing framework FuMi introduces a novel approach to evaluating precision in DL operators during runtime. Researchers in the field of DL testing methodologies can benefit from this innovative perspective, potentially inspiring further advancements and refinements.
- *Practical Utility for Developers.* Developers of DL frameworks can leverage our Pythonic testing scheme, incorporating Fuzz and Check Layers into their testing routines. This provides a practical tool for detecting precision issues during training, enabling more robust model development.
- *Enhanced Operator Precision Understanding.* Our approach empowers DL model trainers with the means to estimate precision loss caused by operators during the training process. This enhanced understanding allows practitioners to identify and address precision-related challenges, contributing to overall model improvement.
- *Reduced Reliance on Golden Data.* Our framework addresses a common limitation in testing approaches by reducing the reliance on golden data. This alleviates the burden on testers, streamlining the testing process and making it more practical for DL framework users.
- *Open Source Code and Data Availability.* We contribute to the research community by making our code and data publicly available. This transparency not only promotes reproducibility but also encourages collaboration and further refinement of our testing framework by the broader research community.

These implications collectively underscore the practical and theoretical contributions of our study, aiming to advance the field of DL testing and enhance the development lifecycle of DL models.

10 Conclusion

Our research introduces FuMi, a groundbreaking precision testing framework for DL. FuMi incorporates Fuzz and Check Layers to address runtime challenges, providing a lightweight, versatile solution for operator evaluation without heavy reliance on golden data.

FuMi eliminates the need for extensive golden data, a common limitation in existing testing methodologies. Instead, it facilitates precision issue detection by incorporating Fuzz and Check Layers, ensuring ease of use and applicability to diverse operators and layers.

Extensive experiments across various models and operators demonstrate FuMi's effectiveness. The framework showcases potential in uncovering precision issues that might otherwise remain undetected. FuMi's open availability of code and data fosters transparency and collaboration within the research community.

In summary, our research addresses current testing challenges and propels future advancements in the dynamic field of DL. FuMi's contributions inspire further research, discussions, and collaborations, fostering a more reliable and precise DL ecosystem.

Acknowledgments

We would like to thank Professor Fabrizio Pastore for his insightful suggestions and constructive feedback, which provided valuable perspectives during our discussions.

We also extend our gratitude to Professor Baowen Xu for his valuable advice during the revision process, which helped us improve the overall quality of the article.

References

- [1] Peng Zhang. 2024. FuMi GitHub repository. Retrieved from <https://github.com/zhangpengNJU/FuMi2024>
- [2] Martin Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2015. TensorFlow: Large-scale machine learning on heterogeneous systems. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI '16)*. USENIX Association, Savannah, GA, USA, 265–283.
- [3] François Chollet. 2018. Keras: The Python deep learning library. *Journal of Open Source Software* 3, 24 (2018), 3370.
- [4] NVIDIA Corporation. 2023. NVIDIA Official Website. Retrieved from <https://www.nvidia.com>
- [5] Daniel Hanchen. 2024. RoPE Loses Precision for Llama/Gemma + Gemma logits.float(). Retrieved from <https://github.com/huggingface/transformers/pull/29285>
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS)*. Retrieved from <https://arxiv.org/abs/2205.14135>
- [7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2023. FlashAttention-2: Faster attention with better parallelism and work partitioning. arXiv:2307.08691. Retrieved from <https://arxiv.org/abs/2307.08691>
- [8] Yinlin Deng, Chenyuan Yang, Anjiang Wei, and Lingming Zhang. 2022. Fuzzing deep-learning libraries via automated relational API inference. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, 44–56. DOI: <https://doi.org/10.1145/3540250.3549085>
- [9] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. Deep Learning. MIT Press. Retrieved from <https://www.deeplearningbook.org/>
- [10] Alex Graves. 2013. Generating sequences with recurrent neural networks. arXiv:1308.0850. Retrieved from <https://arxiv.org/abs/1308.0850>
- [11] Jianmin Guo, Yu Jiang, Yue Zhao, Quan Chen, and Jianguang Sun. 2018. DLFuzz: Differential fuzzing testing of deep learning systems. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 739–743.
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.
- [13] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Identity mappings in deep residual networks. In *Proceedings of the 14th European Conference on Computer Vision (ECCV '16)*, Part IV. Springer, 630–645.
- [14] S. Hochreiter and J. Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9, 8 (1997), 1735–1780.

- [15] Qiang Hu, Yuejun Guo, Xiaofei Xie, Maxime Cordy, Mike Papadakis, Lei Ma, and Yves Le Traon. 2023. Aries: Efficient testing of deep neural networks via labeling-free accuracy estimation. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1776–1787.
- [16] Huawei Technologies Co., Ltd. 2023. Huawei Ascend Official Website. Retrieved from <https://www.hiascend.com/en/>
- [17] Huawei Technologies Co., Ltd. 2024. Dump Mode. Retrieved from https://www.hiascend.com/document/detail/zh/canncommercial/80RC1/devaids/auxiliarydevtool/atlasatc_16_0112.html
- [18] Diederik P. Kingma and Max Welling. 2013. Auto-encoding variational bayes. arXiv:1312.6114. Retrieved from <https://arxiv.org/abs/1312.6114>
- [19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2017. ImageNet classification with deep convolutional neural networks. *Communications of the ACM* 60, 6 (2017), 84–90.
- [20] Cambricon Technologies Corporation Limited. 2023. Cambricon Official Website. Retrieved from <https://www.cambricon.com/>
- [21] Jiawei Liu, Yuheng Huang, Zhijie Wang, Lei Ma, Chunrong Fang, Mingzheng Gu, Xufan Zhang, and Zhenyu Chen. 2023. Generation-based differential fuzzing for deep learning libraries. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2023), 1–28.
- [22] NVIDIA Corporation. 2023. NVIDIA cuBLAS Library. Retrieved from <https://docs.nvidia.com/cuda/cublas/index.html>
- [23] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*. Curran Associates, Inc., 8024–8035.
- [24] PyTorch. 2024. PyTorch Examples Repository. Retrieved December 11, 2024 from <https://github.com/pytorch/examples>
- [25] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.
- [26] S. Varrette, H. Cartiaux, S. Peter, E. Kieffer, T. Valette, and A. Olloh. 2022. Management of an academic HPC & research computing facility: The ULHPC experience 2.0. In *Proceedings of the 6th ACM High Performance Computing and Cluster Technologies Conference (HPCCT '22)*. ACM.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 30.
- [28] Kairui Wang, Yawen Wang, Junjie Wang, and Qing Wang. 2023. Fuzzing with sequence diversity inference for sequential decision-making model testing. In *Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 706–717.
- [29] Anjiang Wei, Yinlin Deng, Chenyuan Yang, and Lingming Zhang. 2022. Free lunch for testing: Fuzzing deep-learning libraries from open source. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 995–1007. DOI : <https://doi.org/10.1145/3510003.3510041>
- [30] Chenyuan Yang, Yinlin Deng, Jiayi Yao, Yuxing Tu, Hanchi Li, and Lingming Zhang. 2023. Fuzzing automatic differentiation in deep-learning libraries. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 1174–1186. DOI : <https://doi.org/10.1109/ICSE48619.2023.00105>
- [31] Xufan Zhang, Jiawei Liu, Ning Sun, Chunrong Fang, Jia Liu, Jiang Wang, Dong Chai, and Zhenyu Chen. 2021. Duo: Differential fuzzing for deep learning operators. *IEEE Transactions on Reliability* 70, 4 (2021), 1671–1685.
- [32] Xufan Zhang, Ning Sun, Chunrong Fang, Jiawei Liu, Jia Liu, Dong Chai, Jiang Wang, and Zhenyu Chen. 2021. Predoo: Precision testing of deep learning operators. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 400–412.

Received 4 July 2024; revised 25 March 2025; accepted 1 May 2025